

Build this project as a curated RSS dashboard hosted on GitHub Pages.

The final workflow must be:

RSS / Atom / YouTube / podcast feeds → Python collector runs automatically with GitHub Actions → collector updates `feed.json` → GitHub Pages dashboard displays the latest items → I manually choose an item and download it as an Obsidian-ready Markdown note.

Use the feed sources already defined in the existing Python script. Keep them editable in one clearly labelled `FEED_SOURCES` list.

Core requirements

1. Python RSS collector

Keep and improve the existing Python collector.

It must:

- Fetch all configured RSS, Atom, YouTube, podcast, science, climate, and geopolitics feeds.
- Extract available metadata:
 - title
 - link
 - publication date
 - author
 - feed title
 - feed URL
 - category
 - kind, such as Article, Video, or Podcast
 - summary or description
 - RSS-provided content where available
 - thumbnail or media image where available
- Clean RSS HTML safely into readable plain text or safe limited HTML.
- Generate one central `feed.json` file.
- Include the newest items from every feed in `feed.json`.
- Do not automatically create individual Obsidian article Markdown files.
- Do not require the user to manually select items before they appear on the dashboard.
- Preserve enough item data in `feed.json` for browser-side Markdown export.
- Include `generated_at` in ISO UTC format.
- Use stable item IDs or fingerprints to avoid duplicate cards where possible.
- Do not hide older current feed entries simply because they were previously seen. `feed.json` should always contain the latest available items from every feed.
- Default to 8 or 10 newest items per feed, configurable through a command-line argument and environment variable.
- Fail gracefully when a feed is unavailable. Include an error field in `feed.json` for failed feeds, but continue processing the rest.

The collector should be runnable locally with:

```
python rss_collector.py
```

Use a `requirements.txt` containing at least:

```
feedparser
```

2. GitHub Actions automation

Create this workflow:

```
.github/workflows/update-rss.yml
```

It must:

- Run automatically every 5 minutes using GitHub Actions cron scheduling.
- Also support manual triggering with `workflow_dispatch`.
- Check out the repository.
- Set up Python.
- Install dependencies from `requirements.txt`.
- Run the collector.
- Commit and push `feed.json` only if it changed.
- Avoid empty commits.
- Set workflow permissions so it can write to repository contents.
- Use the default branch.
- Give the commit a clear message, for example: `Update RSS feed data`.

Do not use an external database or server.

3. GitHub Pages dashboard

Create or improve a static GitHub Pages-compatible site using plain:

- HTML
- CSS
- JavaScript

No framework, backend, database, or build service is required.

The dashboard must:

- Load `feed.json` with:

```
fetch("feed.json", { cache: "no-store" })
```

- Render cards for all articles, videos, and podcasts.
- Show title, source, category, type, publication date, author where available, summary, and thumbnail where available.
- Include filters for:
 - category
 - content type: Article, Video, Podcast
 - source/feed
- Include keyword search across title, author, source, category, and summary.
- Sort newest first by default.
- Include an "Open original" button for each item.
- Include a "Download Obsidian note" button for every item.
- Update data automatically every 60 seconds without fully reloading the page.
- Keep the selected filters and search text during refresh where practical.
- Display:
 - Last checked: HH:MM:SS
 - Feed generated: [generated_at]
- Keep existing content visible if refreshing fails, and show a small non-intrusive error message.
- Do not render RSS content as unsanitized raw HTML.

4. Obsidian Markdown download

When I click "Download Obsidian note", generate a browser download as:

```
YYYY-MM-DD Article title.md
```

Sanitize filenames for Windows-invalid characters:

```
\ / : * ? " < > |
```

Create a reusable JavaScript function such as:

```
downloadArticle(item)
```

It must create a Blob and trigger a local `.md` download.

The downloaded note must have this structure:

```
---
Title: "Article title"
aliases:
```

```

- "Article title"
Date: "ISO publication date"
Type: Article

Source: "[[Source name]]"
feedTitle: "Feed title"
Authors: "[[Author name]]"
Link: "Original URL"
tags:
  - Sources📁/InternetClippings🌐
  - rss
  - rss/category-name
  - rss/feed-name
Status: 📡
Uplink: "[[RSS Notes]]"
---

# Article title

## Summary

> [!warning] Summary
> `INPUT[textArea:Summary]`

## Key points
-
-
-

## Notes
-

## Article content

RSS-provided content, falling back to the RSS summary if full content is
unavailable.

## Related
- [[Dashboard Hub]]
- [[Documents]]
- [[Earthquakes]]
- [[RSS Notes]]
- [[Source name]]

[Open article](original-url)

```

Implement safe YAML escaping for quotes, backslashes, and line breaks.

For `Type`, use the actual item type where possible:

- `Article`
- `Video`
- `Podcast`

5. Project structure

Use or adapt this structure:

```
/
├─ index.html
├─ style.css
├─ app.js
├─ feed.json
├─ rss_collector.py
├─ requirements.txt
├─ README.md
├─ .github/
│   └─ workflows/
│       └─ update-rss.yml
```

6. README

Create or update `README.md` with:

- Local setup instructions
- Python dependency installation
- How to run the collector manually
- How to preview the site locally in VS Code using Live Server or a simple Python HTTP server
- How to enable GitHub Pages from the repository branch
- How to trigger the RSS update workflow manually
- A clear explanation that GitHub Pages is static and GitHub Actions performs the feed collection every 5 minutes
- A clear explanation that downloaded Markdown notes must be moved into my Obsidian vault manually

Please inspect the existing project first and make the changes directly in the appropriate files.

At the end, provide: 1. A concise list of changed files. 2. The final folder structure. 3. Exact local run instructions. 4. Any limitations or feed-specific issues you found.